Machine Learning A (2026)
Home Assignment 6

Casper Moes (hwd362)

Contents

1 Function Basis **2**

2 Sleep Well Revisited **2**

2.1 Principal component analysis 2

3 Differentiable programming **4**

4 Generalization Bounds for Sample Splitting **4**

5 Fusion via Generalization Bounds **5**

1 Function Basis

I use NumPy, scikit-learn and Matplotlib. The data contain 100 curves, each measured at 101 points. I subtract the mean curve before applying PCA, without otherwise rescaling the measurements.

1. I average the 100 curves at each measurement point:

$$\mu(x) = \frac{1}{100} \sum_{k=1}^{100} f^{(k)}(x).$$

The resulting mean curve is shown in Figure 1.

2(a–b). I use the first PCA components as the estimated functions g_i. Four components explain 88.54% of the variance, while five explain 98.09%. We therefore need $F = 5$ to reach 95%. Figure 1 shows the explained variance and the five functions. These capture the main patterns in the curves, but need not equal the original functions used to generate the data.

3. Using the first curve's PCA weights $\widehat{\alpha}_i^{(1)}$, I reconstruct it as

$$\widehat{f}^{(1)}(x) = \mu(x) + \sum_{i=1}^{5} \widehat{\alpha}_i^{(1)} g_i(x).$$

Figure 1 compares this with the measured curve. The reconstruction follows its overall shape while removing much of the noise.

4. The noise has the same variance in every direction. After keeping five components, the remaining 96 directions mostly contain noise. I estimate its variance by averaging their variances:

$$\widehat{\mathrm{Var}}(\epsilon) = \frac{\sum_{i=6}^{101} \lambda_i}{101 - 5} \simeq 0.00964,$$

where λ_i are the covariance eigenvalues, ordered from largest to smallest. Thus the noise variance is about 0.01.

2 Sleep Well Revisited

2.1 Principal component analysis

I use the same software to apply PCA to the training data, which contain 33,724 observations and 16 features. I subtract each feature's mean without rescaling it. Four components explain 89.28% of the variance, while five explain 93.83%. We therefore need 5 components to reach 90%. Figure 2 shows the eigenvalues and the data plotted using the first two components. These two explain 71.54% together. The five classes are shown in different colors; the labels are not used to fit PCA.

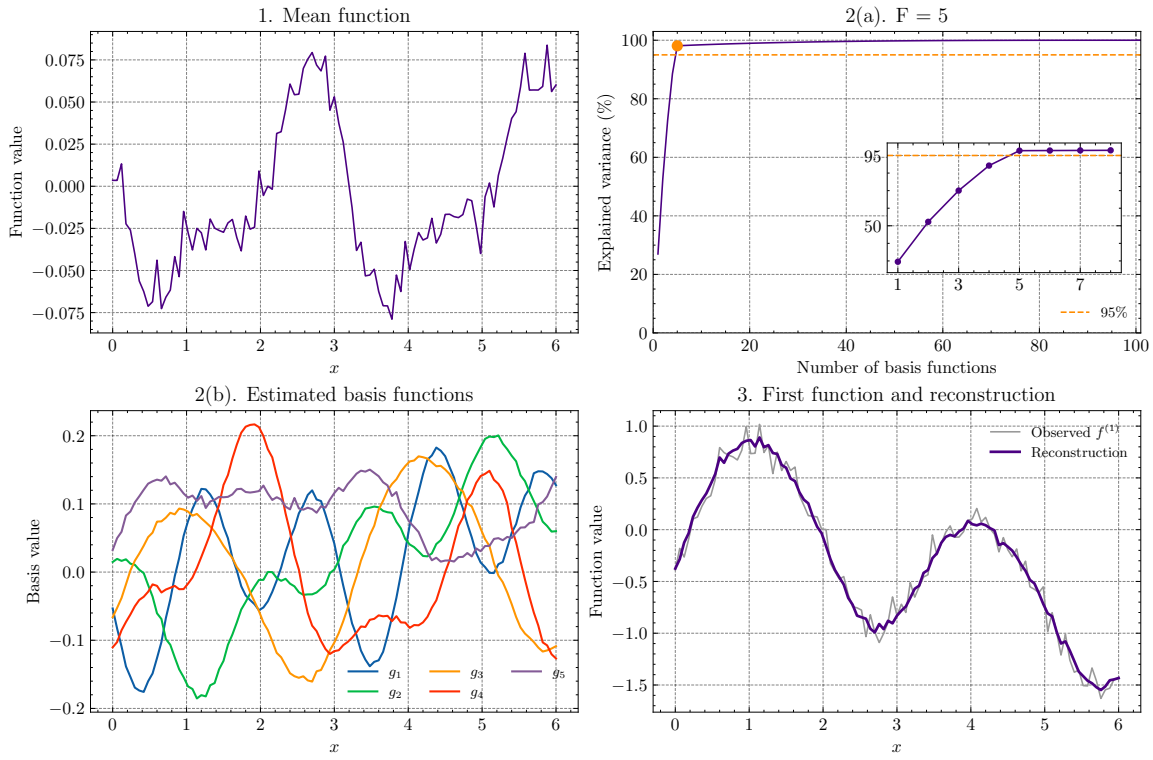


Figure 1: The mean curve, explained variance, five estimated basis functions, and reconstruction of the first curve. The inset shows the first eight components.

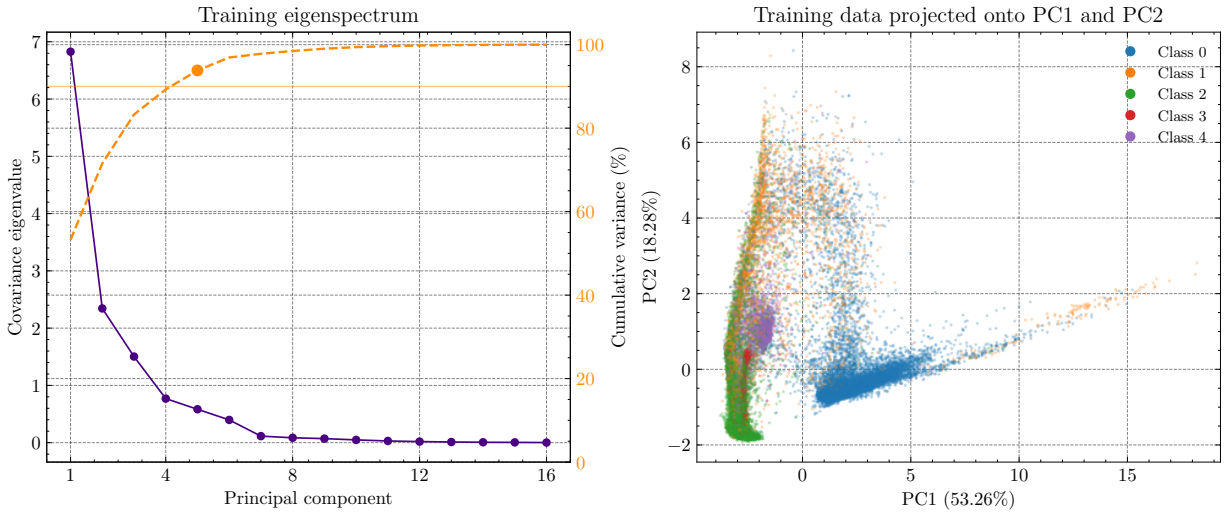


Figure 2: PCA eigenvalues and total explained variance (orange), and the training data shown using the first two components, colored by class.

3 Differentiable programming

I keep the function $f(x, y) = (\alpha x)^2 + y^2 + \alpha xy$, with $\alpha = 0.5$, from the course notebook. I replace the manually calculated gradient and updates with:

```
import torch
alpha, eta, n_iter = 0.5, 0.5, 4
def f(x, y):
    return (alpha*x)**2 + y**2 + alpha*x*y

x = torch.tensor(0.9, dtype=torch.float64, requires_grad=True)
y = torch.tensor(0.8, dtype=torch.float64, requires_grad=True)
optimizer = torch.optim.SGD([x, y], lr=eta)
path = [(x.item(), y.item())]
for _ in range(n_iter):
    optimizer.zero_grad()
    loss = f(x, y)
    loss.backward()
    optimizer.step()
    path.append((x.item(), y.item()))
```

Setting `requires_grad=True` tells PyTorch to calculate gradients for x and y. Their initialization is not part of the calculation being differentiated. Each step, `backward()` calculates the gradient and SGD uses it to update the coordinates. I clear the old gradients with `zero_grad()` and use `item()` to save values for plotting. The four steps end at $(0.280078125, -0.084765625)$, matching the original example. The resulting plot is shown in Figure 3.

4 Generalization Bounds for Sample Splitting

Assume the data are independent observations from the same distribution and the loss is between 0 and 1. The splits are chosen without looking at the observed values, and each model is trained using only its own training set. Let $L(h)$ be its expected loss on new data from the same distribution, and let $0 < \delta < 1$.

For one split, treat the trained model $\hat{h}_i^*$ as fixed. Its test set contains $n_i \geq 1$ independent observations and was not used to train it. Hoeffding's inequality therefore gives

$$\Pr\left(L(\hat{h}_i^*) - \hat{L}(\hat{h}_i^*, S_i^{\text{test}}) > \epsilon_i \mid \hat{h}_i^*\right) \leq e^{-2n_i\epsilon_i^2}.$$

This holds for every fitted model, so averaging over the possible models gives the same bound. To cover all m models, I allow a failure probability of δ/m for each. Setting $\epsilon_i = \sqrt{\ln(m/\delta)/(2n_i)}$ gives, by the union bound,

$$\begin{aligned}\Pr\left(\exists i : \; L(\hat{h}_i^*) > \hat{L}(\hat{h}_i^*, S_i^{\text{test}}) + \epsilon_i\right) \\ \leq \sum_{i=1}^{m} e^{-2n_i\epsilon_i^2} = \sum_{i=1}^{m} \frac{\delta}{m} = \delta.\end{aligned}$$

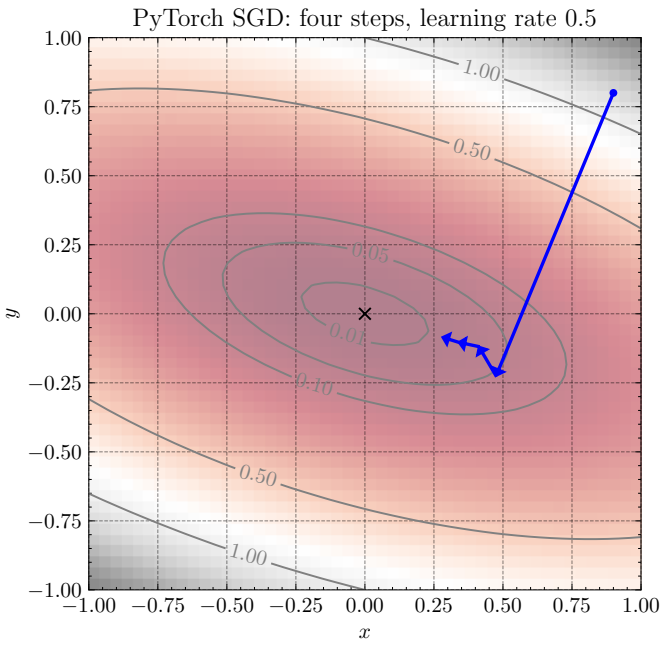


Figure 3: The same four gradient-descent steps as in the original example, now calculated using PyTorch.

Thus, with probability at least $1 - \delta$, all models satisfy

$$L(\hat{h}_i^*) \leq \hat{L}(\hat{h}_i^*, S_i^{\text{test}}) + \sqrt{\frac{\ln(m/\delta)}{2n_i}}.$$

The test sets can overlap because the union bound does not require the failure events to be independent. Since all bounds hold together, we can also use them to choose a model.

5 Fusion via Generalization Bounds

Using Question 4 with $m = 3$ and $\delta = 0.05$, I add $\sqrt{\ln(60)/(2n_i)}$ to each reported test error. This gives the following upper bounds, rounded up:

Classifier	Test size	Test error	Upper bound
1	1000	0.100	0.1453
2	500	0.080	0.1440
3	250	0.075	0.1655

I choose $\hat{h}_2^*$ because it has the smallest upper bound. With probability at least 95%,

$$L(\hat{h}_2^*) \leq 0.1440.$$

The third classifier has a lower test error, but its smaller test set makes that estimate less reliable. The bound from Question 4 covers our choice because it holds for all three models at once.